Year: 2026-2027 **Sem V-NEP**

Course: ASD&D Lab

Experiment 01 - LO2

Date of Performance: 14/07/2026 **Date of Submission: 14/07/2026**

Class: T1

Student Roll No: 2401041 **Student Name:Lakshya Kanjwani**

Aim:To perform version control operations using Git.

Theory

1. A Version Control System (VCS) is a type of software tool designed to help development teams track, manage, and control changes made to source code over time. Git is a distributed version control system, which means that each developer has a complete copy of the repository, including its entire history, stored locally on their machine.

2. The Git workflow consists of several key areas through which files move during the development process:

3. **Working Directory:** The local workspace where developers create, edit, delete, and manage files. Changes made in this area are considered either "untracked" for new files or "modified" for existing files.

4. **Staging Area (Index):** A temporary holding area where developers select and prepare changes that are ready to be included in the next commit. Files are added to this area using the git add command.

5. **Local Repository:** A local storage area within the hidden .git/ directory that maintains the project's history. Changes from the staging area are saved permanently as commits using the git commit command.

6. **Remote Repository:** A hosted copy of the project stored on platforms such as GitHub. It enables developers to back up their work, share code, and collaborate with team members by transferring commits using the git push command.

Commands:

Operation 1: Configuration and Local Repository Initialization

```
PS C:\Users\15L> git config --global user.name "lakshyakanjwanii"
PS C:\Users\15L> git config --global user.email "lakshyakanjwani@gmail.com"
PS C:\Users\15L> mkdir ASDD_Lab_Exp1


    Directory: C:\Users\15L


Mode                 LastWriteTime         Length Name
----                 -------------         ------ ----
d-----         7/14/2026  11:21 AM                ASDD_Lab_Exp1


PS C:\Users\15L> cd ASDD_Lab_Exp1
PS C:\Users\15L\ASDD_Lab_Exp1> git init
Initialized empty Git repository in C:/Users/15L/ASDD_Lab_Exp1/.git/
```

Operation 2: File Creation and Tracking Status Verification

A new file named experiment1.txt containing documentation text was introduced into the working directory.

```
C:\Users\15L\ASDD_Lab_Exp1>echo "Agile Software Development & DevOps - Lab Experiment 1" > experiment1
.txt

C:\Users\15L\ASDD_Lab_Exp1>git status
On branch master

No commits yet

Untracked files:
  (use "git add <file>..." to include in what will be committed)
        experiment1.txt

nothing added to commit but untracked files present (use "git add" to track)
```

Operation 3: Staging Changes to the Index Area

The untracked file was moved out of the working directory and into the Git Staging Area (Index), marking it as ready to be packaged into a commit snapshot.

```
C:\Users\15L\ASDD_Lab_Exp1>git add experiment1.txt

C:\Users\15L\ASDD_Lab_Exp1>git status
On branch master

No commits yet

Changes to be committed:
  (use "git rm --cached <file>..." to unstage)
        new file:   experiment1.txt
```

Operation 4: Committing Changes Locally

The staged content was permanently stored as a cryptographic snapshot inside the local .git repository history with a specific, descriptive message.

```
C:\Users\15L\ASDD_Lab_Exp1>git commit -m "Feat: Initial commit for Experiment 1"
[master (root-commit) bb2c1df] Feat: Initial commit for Experiment 1
 1 file changed, 1 insertion(+)
 create mode 100644 experiment1.txt
```

Operation 5: Linking Remote and Pushing to GitHub

The default local development branch was standardized to main. The local repository was then linked to the remote repository endpoint created on GitHub, and the local history snapshot was uploaded to the server.

```
PS C:\Users\15L\ASDD_Lab_Exp1> git branch -M main
PS C:\Users\15L\ASDD_Lab_Exp1> git remote add origin https://github.com/lakshyakanjwanii/my-project.git
```

```
C:\Users\15L\ASDD_Lab_Exp1>git push -u origin main
info: please complete authentication in your browser...
Enumerating objects: 3, done.
Counting objects: 100% (3/3), done.
Delta compression using up to 20 threads
Compressing objects: 100% (2/2), done.
Writing objects: 100% (3/3), 290 bytes | 290.00 KiB/s, done.
Total 3 (delta 0), reused 0 (delta 0), pack-reused 0 (from 0)
To https://github.com/gurbaxaniishita/ASD-D_Version-control_34.git
 * [new branch]      main -> main
branch 'main' set up to track 'origin/main'.
```

Operation 6: Final Verification of History Logs

A single-line version control log query was executed to guarantee that both the local HEAD indicator and the remote tracking pointers are aligned at the identical history point.

```
PS C:\Users\15L\ASDD_Lab_Exp1> git log --oneline
bb6dcf7 (HEAD -> main) Feat: Initial commit for Experiment 1
```

Operation 7: Modify an existing file

Operation 8: Check difference in working directory (git diff)

See the line-by-line differences of what changed before staging it.

```
PS C:\Users\15L\ASDD_Lab_Exp1> echo "Adding a second line of updates for agile workflows." >> experim
nt1.txt
PS C:\Users\15L\ASDD_Lab_Exp1> git diff
diff --git a/experiment1.txt b/experiment1.txt
index a5693cb..2915244 100644
Binary files a/experiment1.txt and b/experiment1.txt differ
```

Operation 9: View comprehensive system logs (git log)

View the full commit history details including authors, dates, and timestamps.

```
PS C:\Users\15L\ASDD_Lab_Exp1> git log
commit bb6dcf78ce26e3859e703c51a65ab64d327c891f (HEAD -> main)
Author: lakshyakanjwanii <lakshyakanjwani@gmail.com>
Date:   Tue Jul 14 11:25:05 2026 +0530

    Feat: Initial commit for Experiment 1
```

Operation 10: Show commit details for a specific hash (git show)

Examine exactly what happened inside a specific commit snapshot using its first few characters.

```
C:\Users\15L\ASDD_Lab_Exp1>git show bb2c1df
commit bb2c1dfc63cb22fbe348a546bf3657448269e2b9 (HEAD -> main, origin/main)
Author: lakshyakanjwani <lakshyakanjwani@gmail.com>
Date:   Tue Jul 14 10:34:42 2026 +0530

    Feat: Initial commit for Experiment 1

diff --git a/experiment1.txt b/experiment1.txt
new file mode 100644
index 0000000..2381b0a
--- /dev/null
+++ b/experiment1.txt
@@ -0,0 +1 @@
+"Agile Software Development & DevOps - Lab Experiment 1"
```

Branching & Merging Parallel Workflows (Ops 11–16)

Operation 11: Create a new feature branch (git branch)

Create an isolated parallel environment to work on a new feature safely.

Operation 12: List all local branches (git branch -a)

Verify the new branch exists and see which one is currently active (marked with an *).

```
PS C:\Users\15L\ASDD_Lab_Exp1> git branch feature-login
fatal: a branch named 'feature-login' already exists
PS C:\Users\15L\ASDD_Lab_Exp1> git branch -a
  feature-login
* main
```

Operation 13: Switch to the feature branch (git switch)

Move your HEAD pointer over to the newly created branch workspace.

```
C:\Users\15L\ASDD_Lab_Exp1>git switch feature-login
M       experiment1.txt
Switched to branch 'feature-login'
```

Operation 14: Create a new file on the feature branch

```
C:\Users\15L\ASDD_Lab_Exp1>echo "Tracking login feature logic independently." > auth.txt

C:\Users\15L\ASDD_Lab_Exp1>git add auth.txt

C:\Users\15L\ASDD_Lab_Exp1>git commit -m "Feat: Implement login authentication tracking"
[feature-login f702f33] Feat: Implement login authentication tracking
 1 file changed, 1 insertion(+)
 create mode 100644 auth.txt
```

Operation 15: Switch back to the main branch

```
C:\Users\15L\ASDD_Lab_Exp1>git switch main
M       experiment1.txt
Switched to branch 'main'
Your branch is up to date with 'origin/main'.
```

Operation 16: Merge the feature branch into main (git merge)

Bring the changes from the feature branch seamlessly back into your main branch.

```
C:\Users\15L\ASDD_Lab_Exp1>git merge feature-login
Updating bb2c1df..f702f33
Fast-forward
 auth.txt | 1 +
 1 file changed, 1 insertion(+)
 create mode 100644 auth.txt
```

Undoing Mistakes & Advanced State Control (Ops 17–21)

Operation 17: Discard unstaged changes in a file (git checkout --)

Accidentally typed garbage into a file? Let's break a file and restore it to its pristine state.

```
C:\Users\15L\ASDD_Lab_Exp1>echo "Oops this is bad code" >> experiment1.txt

C:\Users\15L\ASDD_Lab_Exp1>git checkout -- experiment1.txt
```

```
C:\Users\15L\ASDD_Lab_Exp1>type experiment1.txt
"Agile Software Development & DevOps - Lab Experiment 1"
```

Operation 18: Unstage a file from the index (git restore --staged)

Stage a file, then change your mind and pull it back out of the staging area without erasing its contents.

```
C:\Users\15L\ASDD_Lab_Exp1>echo "Temporary draft data" > draft.txt

C:\Users\15L\ASDD_Lab_Exp1>git add draft.txt

C:\Users\15L\ASDD_Lab_Exp1>git restore --staged draft.txt

C:\Users\15L\ASDD_Lab_Exp1>git status
On branch main
Your branch is ahead of 'origin/main' by 1 commit.
  (use "git push" to publish your local commits)

Untracked files:
  (use "git add <file>..." to include in what will be committed)
        draft.txt

nothing added to commit but untracked files present (use "git add" to track)
```

Operation 19: Save temporary uncommitted work (git stash)

Save your current messy working directory changes safely to an internal clipboard so you can work on something else with a clean slate.

```
C:\Users\15L\ASDD_Lab_Exp1>echo "Halfway finished feature code" >> experiment1.txt

C:\Users\15L\ASDD_Lab_Exp1>git stash
Saved working directory and index state WIP on main: f702f33 Feat: Implement login authentication trac
king
```

Operation 21 & 22: Enumerate Cache & Pop State

Op 21: List active stashes to verify it's securely stored

```
C:\Users\15L\ASDD_Lab_Exp1>git stash list
stash@{0}: WIP on main: f702f33 Feat: Implement login authentication tracking

C:\Users\15L\ASDD_Lab_Exp1>git stash pop
On branch main
Your branch is ahead of 'origin/main' by 1 commit.
  (use "git push" to publish your local commits)

Changes not staged for commit:
  (use "git add <file>..." to update what will be committed)
  (use "git restore <file>..." to discard changes in working directory)
        modified:   experiment1.txt

Untracked files:
  (use "git add <file>..." to include in what will be committed)
        draft.txt

no changes added to commit (use "git add" and/or "git commit -a")
Dropped refs/stash@{0} (505c16adefc7895079023ff7b0577535a4732e00)

C:\Users\15L\ASDD_Lab_Exp1>
```

Operation 23 & 24: Remote Auditing & Tracking Update

Operation 23: Verify the remote server connection mapping paths

git remote -v

Operation 24: Push all your newly consolidated branch changes to the cloud

git push origin main

```
C:\Users\15L\ASDD_Lab_Expl>git remote -v
origin  https://github.com/lakshyakanjwanii/my-project.git (fetch)
origin  https://github.com/lakshyakanjwanii/my-project.git (push)

C:\Users\15L\ASDD_Lab_Expl>git push origin main
Enumerating objects: 4, done.
Coumerating objects: 4, done.
Counting objects: 100% (4/4), done.
Delta compression using up to 20 threads
Compressing objects: 100% (2/2), done.
Writing objects: 100% (3/3), 349 bytes | 349.00 KiB/s, done.
Total 3 (delta 0), reused 0 (delta 0), pack-reused 0 (from 0)
To https://github.com/lakshyakanjwanii/my-project.git
   bb2c1df..f702f33  main -> main
```

Operation 25: The Ultimate Master Version Check

```
C:\Users\15L\ASDD_Lab_Expl>git log --oneline --graph --all
* f702f33 (HEAD -> main, origin/main, feature-login) Feat: Implement login authentication tracking
* bb2c1df Feat: Initial commit for Experiment 1
```

Conclusion

What I Learned:-

- During my learning journey with Git, I explored the complete process of managing and tracking code changes effectively.
- **Understanding Git Workflow:** I gained knowledge of how files move through Git's three main stages — the **Working Directory**, where changes are created; the **Staging Area**, where updates are prepared; and the **Repository**, where changes are permanently recorded through commits.

- **Using Branches for Development:** I learned how branches allow developers to work on different features or ideas separately while keeping the main project stable and unaffected.

- **Managing Errors and Changes:** I practiced correcting mistakes by undoing unwanted modifications, removing files from staging when necessary, and using **Git Stash** to temporarily save unfinished work without losing progress.

- **Connecting with Remote Repositories:** I learned how to link my local Git environment with GitHub, troubleshoot authentication problems, and push my code to a remote repository for online storage and collaboration.

- **Tracking Changes Over Time:** I learned how to analyze commit history using Git log commands, which helped me understand the sequence of updates and view the complete evolution of a project.